

EL1 中，如图 14.10 所示。

当发生虚拟机下陷时（例如虚拟机执行超级调用），首先下陷到 EL2，这是由硬件虚拟化的机制决定的。而 EL2 中的 KVM Lowvisor 在处理此次虚拟机下陷的过程中可能需要使用 Linux 的功能，所以它将控制流转回到 EL1 的 KVM 里，并调用 Linux 的功能。在此之后，KVM 重新回到 Lowvisor，并最终回到虚拟机内。可以看到，这样的架构设计造成 KVM 和 Lowvisor 之间的多次特权级切换，因而对虚拟机的运行性能带来了一定开销。

为了解决上述性能问题，第二阶段的 ARMv8.1 中进一步扩展了硬件虚拟化的功能，并增加了 VHE（Virtualization Host Extension）。VHE 意味着可在 EL2 中运行完整的宿主操作系统。此时 KVM 的架构也发生了相应转变，如图 14.11 所示。因此，在 ARMv8.1 之后，KVM 和宿主操作系统 Linux 的交互都不会带来任何特权级切换，从而优化了虚拟化的性能。

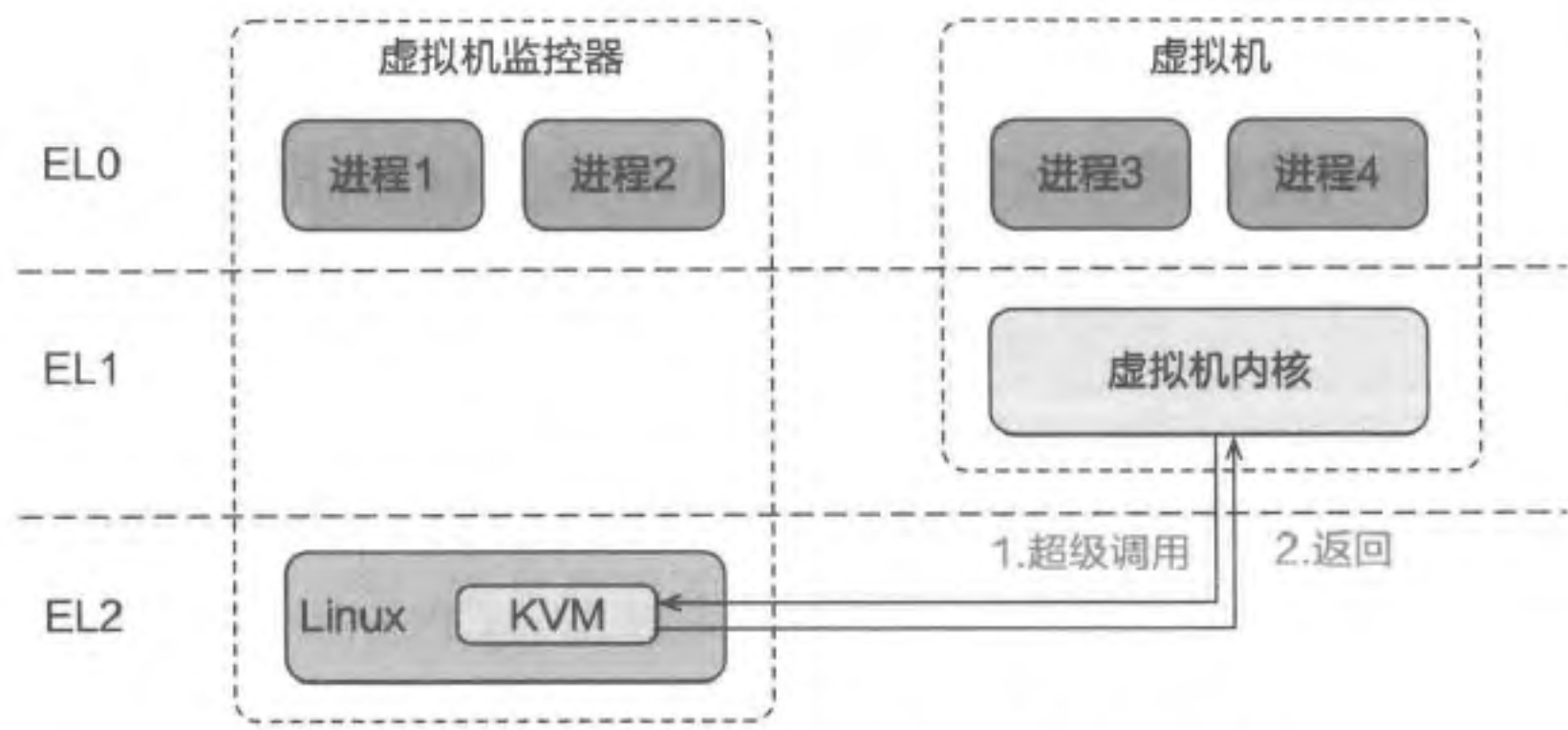


图 14.11 ARMv8.1 VHE 硬件虚拟化扩展

除了上述功能外，硬件虚拟化还对内存虚拟化提供了相关支持。我们将在下一节详细介绍。

14.3.7 小结

本节介绍了实现 CPU 虚拟化的 5 种技术，这些技术可以分为两大类，如表 14.3 所示。在硬件虚拟化技术出现之前，CPU 虚拟化技术主要通过软件技术解决不可虚拟化架构带来的问题，而在硬件虚拟化技术出现之后，CPU 虚拟化技术主要依托于硬件提供的虚拟化功能，例如通过配置 HCR_EL2 寄存器控制虚拟机的下陷行为等。

表 14.3 CPU 虚拟化技术总结

软件技术	硬件虚拟化技术
解释执行	新的硬件特权级，所有敏感指令可下陷
动态二进制翻译	
扫描 - 翻译	
半虚拟化技术	

上述技术在性能以及是否需要修改虚拟机代码等方面都存在着各自的优缺点，因此

有着相应的适用范围。此外，在硬件虚拟化技术推出之后，曾经的软件技术也未过时，依然存在适用场景。首先，当物理主机 ISA 和虚拟机 ISA 不同时，将不能使用硬件虚拟化技术，只能选择软件技术，例如在 x86 的机器上模拟 ARM 虚拟机。其次，在使用硬件虚拟化技术之后，依然可以组合使用某些软件技术，例如可以使用半虚拟化技术优化 I/O 场景下的性能，我们将在 14.5 节进行介绍。

14.4 内存虚拟化

本节主要知识点

- 为什么需要内存虚拟化？
- 内存虚拟化中的三种内存地址分别是什么？
- 如何通过影子页表和直接页表映射机制实现内存虚拟化？
- AArch64 架构中的两阶段地址翻译机制如何工作？
- 基于内存虚拟化的换页机制如何实现？

为什么需要内存虚拟化？我们首先从没有系统虚拟化的场景说起。使用系统虚拟化之前，操作系统内核直接运行在最高特权级，因此它有权限管理整个物理地址空间，“看到的”的是从零地址开始连续增长的物理地址空间。使用系统虚拟化之后，一台物理主机上可同时运行多台虚拟机，此时每个客户操作系统“看到的”物理地址空间应该从零开始连续增长（否则虚拟机无法正常运行）。此外，虚拟机监控器不允许客户操作系统访问不属于它的物理内存区域。如果此时某个客户操作系统可以访问任意物理内存区域，它就能读取甚至修改其他虚拟机以及虚拟机监控器内存中的数据，这破坏了虚拟机的内存隔离，严重威胁整个系统的安全。

因此，内存虚拟化需要满足两个需求：

- 第一，为每台虚拟机提供从零地址开始连续增长的物理地址空间。
- 第二，实现虚拟机之间的内存隔离，每台虚拟机只能访问分配给它的物理内存区域。

该如何满足这两个需求？这里介绍一种全新类型的内存地址——客户物理地址，这是虚拟机内使用的物理地址，不是真实的在总线中访存的地址。真正访存的物理地址改称为主机物理地址。在虚拟机的执行过程中，需要将客户物理地址转换成主机物理地址，然后通过后者访问存储在内存中的数据和代码。相应地，虚拟机监控器需要提供一种新的地址翻译机制，将客户物理地址翻译成主机物理地址。如图 14.12 所示，到目前为止，我们接触了三种不同类型的地址：

- 第一种是进程和客户操作系统使用的客户虚拟地址（Guest Virtual Address，

- GVA)，即第 4 章介绍的虚拟地址。
- 第二种是客户操作系统管理的客户物理地址（Guest Physical Address, GPA）。
 - 第三种是 CPU 发送至总线进行访存的主机物理地址（Host Physical Address, HPA）。

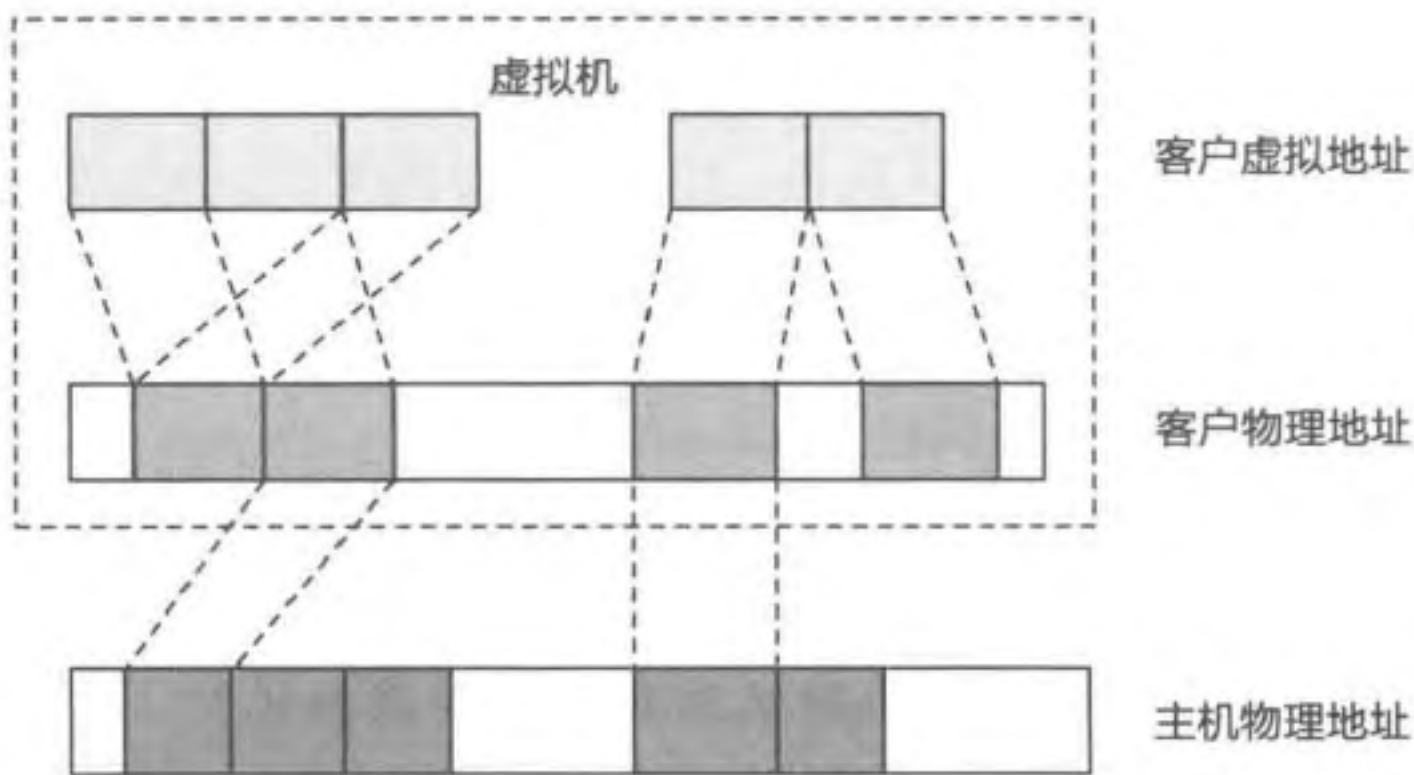


图 14.12 虚拟化场景下三种不同类型的内存地址

在本章中，客户虚拟地址转换为客户物理地址的过程称为第一阶段地址翻译，客户物理地址转换为主机物理地址的过程称为第二阶段地址翻译。

引入客户物理地址的概念之后，内存虚拟化技术拥有以下四个优点。第一，可为每个虚拟机提供一个从零地址开始连续增长的客户物理地址空间，客户操作系统“以为”自己仍然在管理物理地址空间。第二，第二阶段地址翻译可提高主机物理内存的使用效率，例如，虚拟机监控器将不连续的主机物理地址内存映射成连续的客户物理地址，从而减少内存碎片。第三，即使多个虚拟机请求的客户物理内存总数超出主机物理内存的大小，也可通过内存虚拟化机制同时支持多台虚拟机的执行，我们将在 14.4.4 节详细介绍这项技术。第四，通过内存虚拟化技术可实现虚拟机之间客户物理地址空间的安全隔离，因为虚拟机监控器借助第二阶段地址翻译过程限制了每一台虚拟机的地址范围，任何虚拟机不能读写其他虚拟机中的物理内存区域（除了共享内存的情况外）。

14.4.1 影子页表机制

正如上一节所说，客户虚拟地址需要经过两个阶段的地址翻译，才可转换到主机物理地址。在硬件虚拟化技术出现之前，MMU 中只有一个页表。在虚拟化环境内，该页表已经被运行在 EL1 的虚拟机监控器所使用。然而，此时客户操作系统也需要使用页表实现第一阶段地址翻译。如何在只有一个页表的情况下同时实现两种地址翻译？

在介绍具体的解决方案之前，我们先回顾一下非虚拟化场景下页表的使用方式。图 14.13 展示了在操作系统内核中页表使用的三个阶段。第一个阶段是静态配置，在进程被正式调度之前，操作系统内核首先为此进程静态配置一个页表，此页表中维护了

从虚拟地址到物理地址的映射关系。第二个阶段是页表安装，在决定调度进程之后，操作系统内核将此进程的页表基地址写入页表基地址寄存器（如 TTBR0_EL1）。第三个阶段是动态翻译，硬件 MMU 按照 TTBR0_EL1 指向的页表，动态地将虚拟地址翻译成物理地址，并完成内存访问。动态翻译的过程完全由 MMU 完成，不需要操作系统内核的参与。

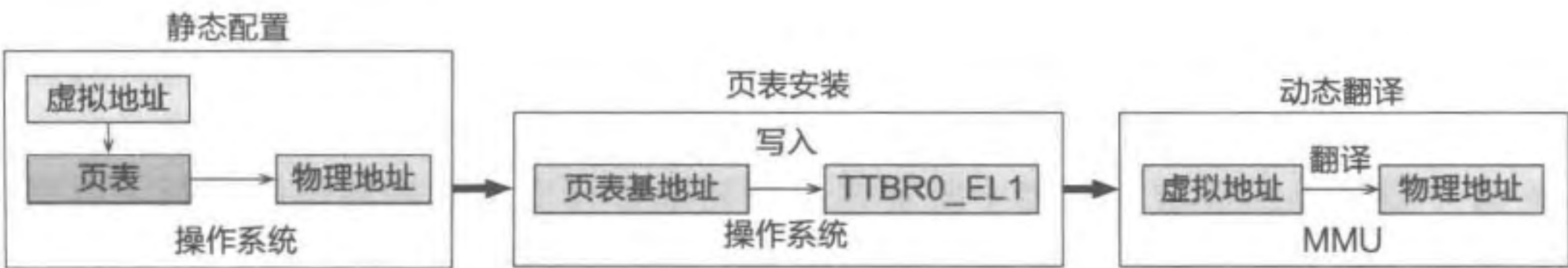


图 14.13 页表使用的三个阶段

如果虚拟机监控器在静态配置阶段将页表中客户虚拟地址到客户物理地址的映射改成客户虚拟地址直接到主机物理地址的映射，那么在动态翻译时，即使 MMU 中只有一个页表，也可完成两个阶段的地址翻译。图 14.14 展示了这个过程。在操作系统写入 TTBR0_EL1 时发生虚拟机下陷，此时虚拟机监控器根据页表内容创建一个新的页表。新页表的映射与虚拟机安装的页表很相似，只是将其中的客户物理地址改写成主机物理地址。创建完新页表后，虚拟机监控器将新页表基地址写入 TTBR0_EL1。在这之后，MMU 根据页表中记录的映射，直接将客户虚拟地址翻译成主机物理地址。这就是影子页表技术。

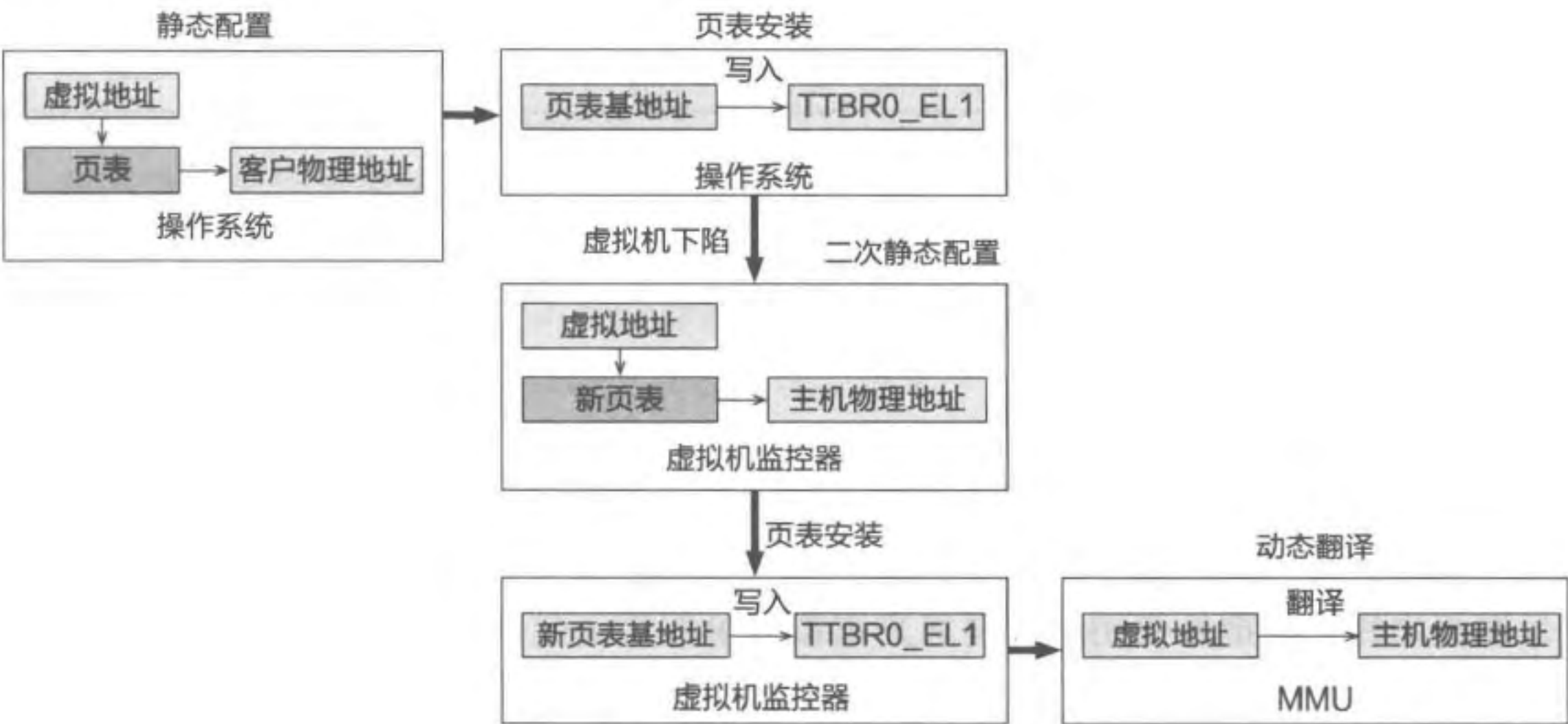


图 14.14 页表使用的三个阶段

影子页表（Shadow Page Table, SPT）是相对于虚拟机内使用的页表而言的概念。虚

拟机中的操作系统只能为进程（或内核）配置页表，它也会将页表安装到 MMU 中，“影子”的含义表现在两个层面。首先，影子页表是虚拟机监控器为虚拟机“秘密”配置的一个页表，此页表不为虚拟机所见，对其完全透明，更不能被虚拟机修改。其次，影子页表的内容与客户虚拟机使用的页表高度相关，它的映射内容随着页表内容的改变而改变，就像“影子”一样。

我们该如何创建影子页表？影子页表机制的使用，必须与 CPU 虚拟化的相关技术相结合。例如，通过软件模拟方式使得敏感指令（写入 TTBR0_EL1）引起下陷。具体创建步骤如下：

- 第一步：操作系统在页表中配置客户虚拟地址到客户物理地址的映射，由于虚拟机内只能使用客户物理地址，所以它将此页表的客户物理地址写入 TTBR0_EL1。
- 第二步：虚拟机监控器通过软件模拟的方式使得系统寄存器写入操作引发虚拟机下陷。
- 第三步：创建影子页表的流程见代码片段 14.1，具体来说，虚拟机监控器需要为每个虚拟机维护一个地址转换表（address_translation_table），其中记录了客户物理地址到主机物理地址的映射关系。虚拟机监控器遍历客户操作系统需要安装的页表，并根据表中内容创建一个影子页表，其内容与客户页表一一对应，只是将客户物理地址改写成主机物理地址。影子页表最终包含客户虚拟地址到主机物理地址的映射关系。
- 第四步：虚拟机监控器通过配置影子页表，为原页表所在的内存设置只读权限。
- 第五步：影子页表的主机物理地址被虚拟机监控器写入 TTBR0_EL1 寄存器。
- 第六步：虚拟机监控器恢复虚拟机的执行。

代码片段 14.1 生成影子页表的伪代码

```

1 set_TTBR0_EL1(guest_page_table):
2   for GVA from 0 to MAX_GVA
3     if guest_page_table[GVA] & IS_VALID:
4       GPA = guest_page_table[GVA] >> 12           // 客户页表
5       HPA = address_translation_table[GPA] >> 12   // 地址转换表
6       shadow_page_table[GVA] = (HPA << 12) | PTE_P // 影子页表
7     else
8       shadow_page_table[GVA] = 0
9   set guest_page_table to READ_ONLY
10  TTBR0_EL1 = HOST_PHYSICAL_ADDR(shadow_page_table)

```

在此之后，虚拟机使用的任何客户虚拟地址都将经过影子页表被直接翻译到主机物理地址。

我们在上述第四步中已经将原页表所在的内存设置为只读权限。如果客户操作系统修改页表，将触发一次缺页异常，并下陷至虚拟机监控器。虚拟机监控器根据触发异常的客户虚拟地址，发现虚拟机试图修改页表。它将模拟此次修改操作，并将修改后的结

果同步到影子页表中。同理，如果客户操作系统试图更换页表，将再次触发上述建立影子页表的过程。

影子页表的缺页异常处理流程

虚拟机运行在用户态，一旦发生缺页异常，首先会下陷到内核态并唤醒虚拟机监控器注册的缺页异常处理函数。之后，虚拟机监控器查看引起此次下陷的客户虚拟地址并查询客户页表，如果客户页表中不存在与下陷的客户虚拟地址相关的映射，或者存在映射但是权限不够，那么虚拟机监控器将这次缺页异常插入客户操作系统，直接调用客户操作系统注册的缺页异常处理函数。

如果客户页表中存在相关页表项且权限足够，这一次缺页异常则是由影子页表未与客户页表同步造成的。因此，虚拟机监控器需要将客户页表中的权限同步至影子页表中，再恢复虚拟机的执行^①。

虚拟机内的地址隔离

由于虚拟机内的内核态与用户态共享同一份影子页表，它们在影子页表内的虚拟地址都映射成用户态可访问。这种映射方式意味着虚拟机内的用户态进程可以访问客户操作系统的内存数据，从而打破虚拟机内核态和用户态之间的内存隔离。为了阻止这种情况的发生，虚拟机监控器为虚拟机内核与用户态进程维护两个不同的影子页表。虚拟机内核的影子页表中包含对应用户态进程以及内核的所有地址映射，而用户态进程使用的影子页表中不含有内核的地址范围，只包含此进程的地址映射。当发生虚拟机内的内核态与用户态切换时，虚拟机监控器会安装好对应的影子页表。

影子页表机制的优点是地址翻译速度快。由于影子页表中记录了客户虚拟地址到主机物理地址的直接映射，因此 MMU 只需遍历一个页表就可完成两个阶段的地址翻译过程。即使发生 TLB 不命中的情况，MMU 只需要遍历一遍影子页表，最多只需要 4 次内存访问操作就可以完成地址翻译过程。

影子页表机制有两个缺点。首先，影子页表的建立和后续每次更新都需要虚拟机监控器的介入，这不仅会为虚拟机监控器增加实现复杂度，也会带来较大的性能开销。其次，影子页表与页表一一对应，因此虚拟机监控器需要为每一个进程维护相对应的影子页表，也需要为内核维护单独的影子页表，这会带来一定的内存开销。

14.4.2 直接页表映射机制

影子页表技术的复杂性源于虚拟机监控器希望透明地提供虚拟机抽象。因此，虚拟机监控器需要不断捕捉客户操作系统对页表的修改，并将其同步至影子页表中。如果采用一种不同的思路，不再“费力”透明地提供虚拟机抽象，而是让虚拟机“知道”自己运行在虚拟机监控器之上，则可简化内存虚拟化的设计与实现。这就是直接页表（Direct

^① 然而实际的处理情况涉及访问位和修改位的同步问题，比简单的权限同步更为复杂，这里不再赘述。

Paging) 映射机制, 也是半虚拟化思想在内存虚拟化方面的应用。

与影子页表机制相比, 直接页表映射机制主要有以下两个重要特点:

- 只有一个页表: 在直接页表映射机制中不存在影子页表的概念, 虚拟机内维护的页表将直接被安装在硬件 MMU 中, 并用来进行后续的地址翻译。因此, 这里不需要使用客户物理地址, 客户操作系统在页表中记录了客户虚拟地址到主机物理地址的映射关系。为了方便客户操作系统配置页表映射, 虚拟机监控器需要告知虚拟机允许使用的主机物理地址范围。
- 调用接口进行修改: 页表中的映射对系统安全有着重要影响, 如果允许虚拟机直接维护页表映射, 虚拟机可能会恶意或无意地在页表中添加非法映射, 从而可以访问其他虚拟机甚至虚拟机监控器的内存区域。为了防御这种攻击, 直接页表映射机制将虚拟机的页表页设置为只读权限, 从而阻止虚拟机直接修改页表页。虚拟机必须使用虚拟机监控器提供的超级调用接口进行修改。在接收到超级调用请求后, 虚拟机监控器将检查此次需要添加或修改的映射是否合法, 例如是否使用了其他虚拟机的主机物理地址。如果不合法, 将拒绝此次修改。

直接页表映射机制具有以下优点。首先, 由于不需要维护影子页表, 虚拟机监控器的内存虚拟化模块的实现复杂度将会降低。其次, 影子页表机制通过大量缺页异常将虚拟机对客户页表的修改“透明”地同步到影子页表中, 这会带来较大的性能开销; 而在直接页表映射机制中, 由于虚拟机可将对页表的多次修改整合成一次超级调用, 因此可实现批量处理的效果并带来一定的性能提升。然而, 直接页表映射机制也存在半虚拟化技术本身的缺点, 例如需要修改客户操作系统代码。

14.4.3 两阶段地址翻译机制

我们已经了解了影子页表和直接页表映射机制, 这些机制可以在硬件中只有一个页表时实现内存虚拟化。本节将介绍如何通过硬件虚拟化技术实现内存虚拟化。

在 ARM 硬件虚拟化拓展中, 硬件在 EL2 特权级添加了第二阶段页表 (Stage-2 Page Table)。此页表记录客户物理地址到主机物理地址的映射关系。原先虚拟机内使用的页表被称为第一阶段页表 (Stage-1 Page Table), 记录客户虚拟地址到客户物理地址的映射关系。我们在介绍影子页表机制时提到, 虚拟机监控器为每个虚拟机维护一个地址转换表, 此表也维护了客户物理地址与主机物理地址的映射关系。影子页表机制中的地址转换表仅仅起着信息维护的作用, 在硬件 MMU 翻译时不起作用。与此不同的是, EL2 中的第二阶段页表可被物理 MMU 识别, 并参与 MMU 的地址翻译过程。

图 14.15 展示了第二阶段页表的组织形式, 该页表最多由 4 级页表页构成。最高级称为 L0, 最低级 (指向最终物理页) 称为 L3。每一级页表页为 4KB, 包含 512 个页表项, 每一项为 8 字节。每一个有效的页表项存储下一级页表页或内存页的主机物理地址, 也存储相关的权限。在 ARM 硬件虚拟化中, 虚拟机监控器在运行虚拟机之前, 首

先需要将此虚拟机的第二阶段页表页基地址（主机物理地址）写入 VTTBR_EL2 寄存器中，并将 HCR_EL2 系统寄存器的第 0 位（VM 位）设置为 1，这表示打开虚拟机中的第二阶段页表翻译机制。在虚拟机执行的过程中，任何客户物理地址都被硬件 MMU 通过 VTTBR_EL2 指向的第二阶段页表翻译成对应的主机物理地址，整个翻译过程不需要任何虚拟机监控器的介入。

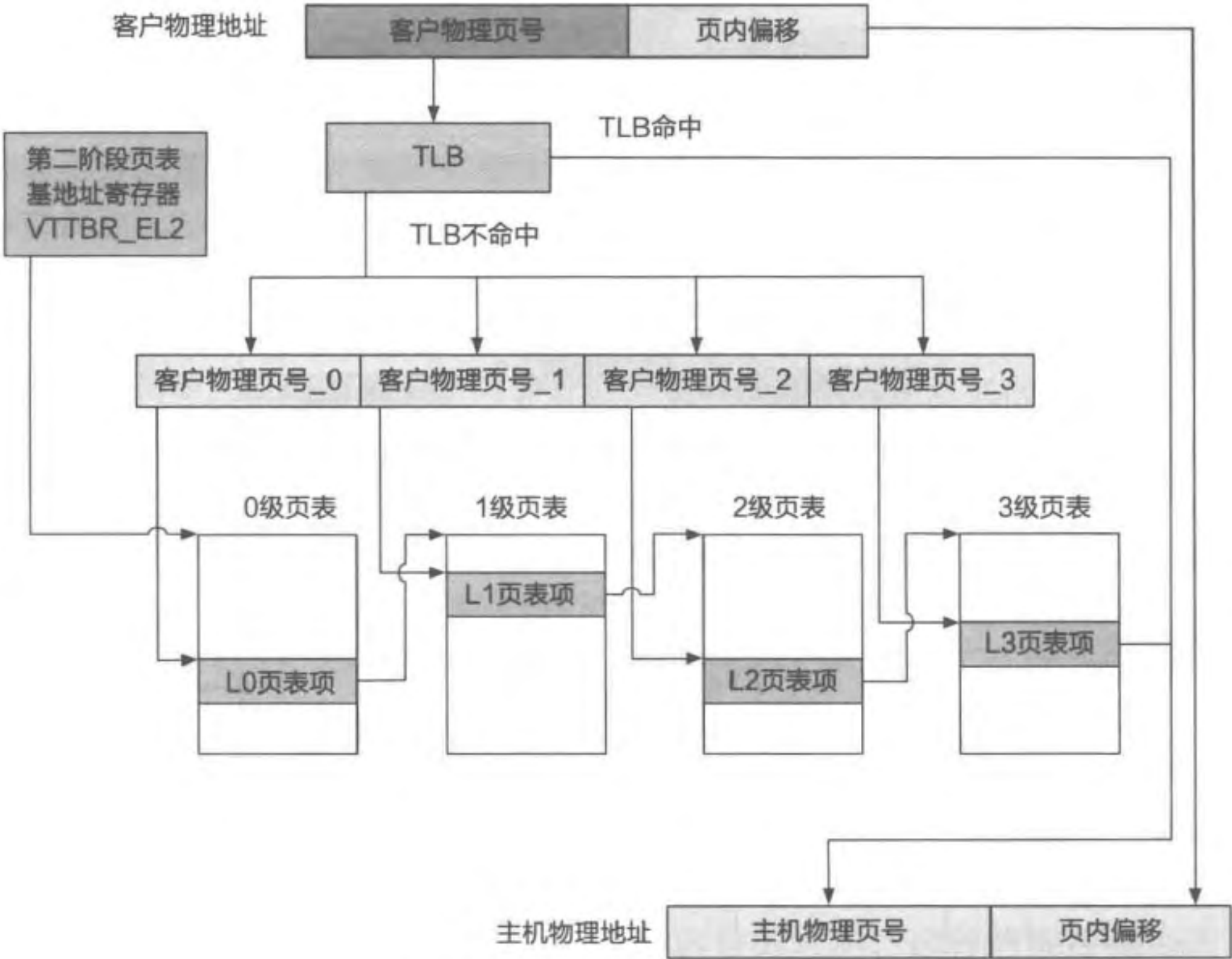


图 14.15 第二阶段页表的组织形式

下面我们详细介绍两阶段的页表如何共同参与地址翻译过程，并将客户虚拟地址翻译成最终的主机物理地址。完整的翻译过程如图 14.16 所示。首先，为了翻译客户虚拟地址，硬件 MMU 需要读取 TTBR0_EL1（或 TTBR1_EL1）寄存器，找到第一阶段页表的基地址。请注意，这个基地址是一个客户物理地址。我们已经知道，任何客户物理地址都必须经过第二阶段页表的翻译。因此，硬件 MMU 将此客户物理地址通过第二阶段页表翻译，得到 L0 页表页的主机物理地址。MMU 再由此主机物理地址进行访存操作并读取第一阶段页表的 L0 页表页的内容。MMU 之后根据客户虚拟地址相应的偏移量，定位第一阶段页表页中相关的项，并得到下一级（L1）页表页的客户物理地址。类似地，MMU 将此客户物理地址翻译成对应的主机物理地址，并得到 L1 页表页的内容。依次类推，4 级页表都会经过这样的翻译过程。L3 页表项中记录了最终内存页的客户物理地址。

此地址依然需要经历第二阶段页表的翻译，得到此客户物理地址对应的主机物理地址并实现访存。在发生 TLB 不命中时，翻译一次客户虚拟地址需要经过最多 24 次访存操作。

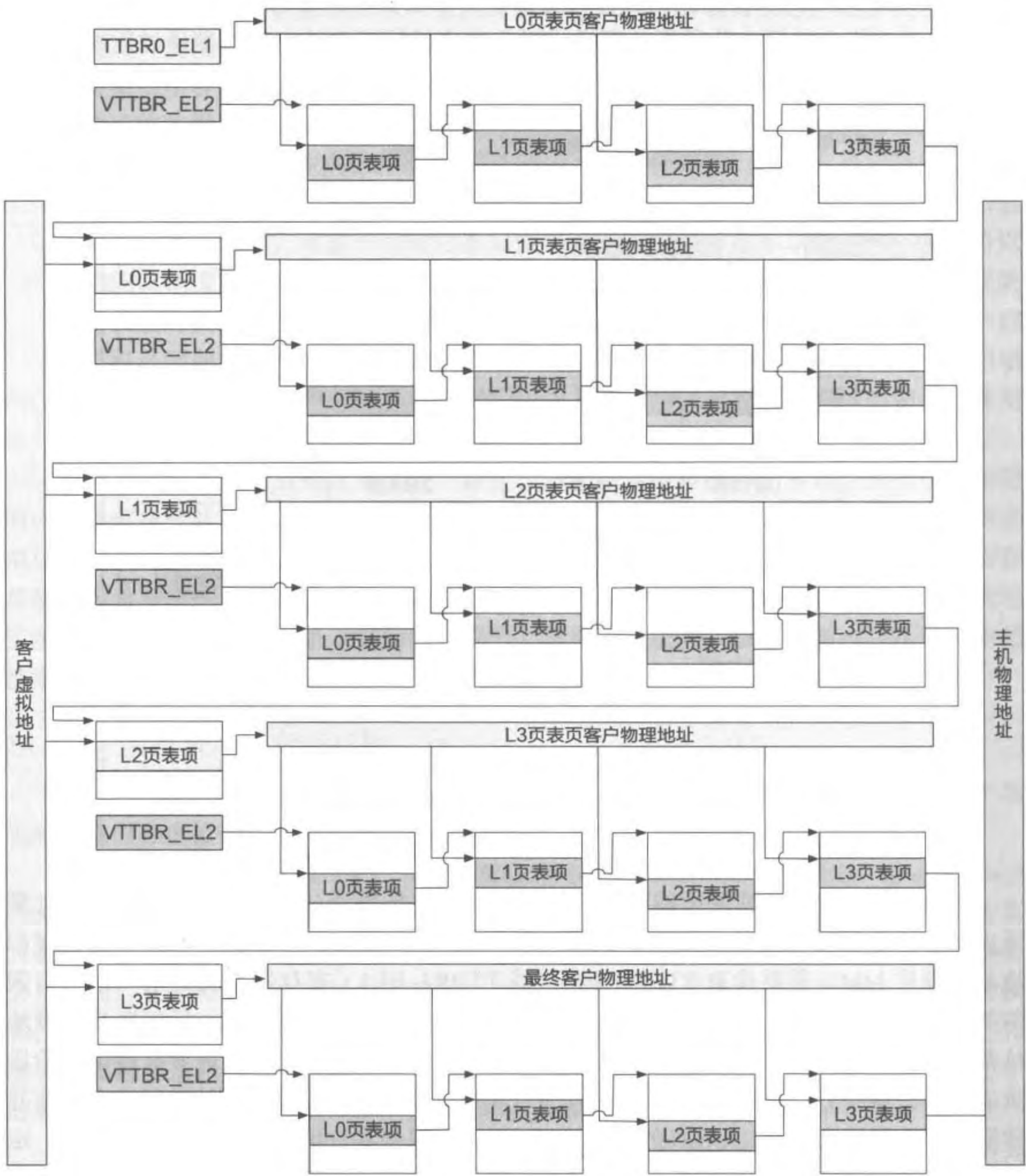


图 14.16 两阶段页表的地址翻译过程

在使用第二阶段页表翻译技术时，虚拟机内缺页异常的处理将会分为两种情况。第

一，在客户虚拟机中发生的任何缺页异常不再引起任何虚拟机下陷，硬件将会直接调用客户虚拟机注册的缺页异常处理函数，待处理完之后将恢复执行。第二，发生第二阶段页表相关的缺页异常之后会产生虚拟机下陷，硬件调用虚拟机监控器注册的缺页异常处理函数，虚拟机监控器此时检查引起异常的客户物理地址以及相关权限，确认是否未添加映射或权限不够，并根据具体的情况进行处理。虚拟机监控器处理完成后将恢复虚拟机的正常执行。

硬件虚拟化的第二阶段页表翻译技术有三个优点。首先，第一阶段页表和第二阶段页表将分开维护。客户操作系统在更新页表时不会引起任何虚拟机下陷，因此相对影子页表机制而言，页表的更新性能更好。其次，虚拟机监控器为虚拟机配置的第二阶段页表在虚拟机运行时起作用，不需要为每一个进程单独配置一个页表。因此，这项技术的内存开销较小。最后，在影子页表中，每次缺页异常都下陷到虚拟机监控器，由它检查引起异常的地址之后再决定是虚拟机还是监控器处理。在第二阶段页表翻译技术中，由第一阶段页表和第二阶段页表引起的异常将分别唤醒虚拟机和虚拟机监控器，大大提高了缺页异常的处理性能。

正如我们之前看到的，发生 TLB 不命中时，在最差的情况下，一次客户虚拟地址的翻译将会经过 24 次内存访问，这会带来巨大的性能开销。因此，ARM 体系结构针对 TLB 技术做了相关的优化，使得 TLB 可以直接缓存客户虚拟地址到主机物理地址的映射。

14.4.4 换页和气球机制

假设一台物理主机上的物理内存大小为 48GB。如果需要在这台物理主机上同时运行 4 台虚拟机，其中每一台虚拟机的内存需求为 24GB。这 4 台虚拟机的内存需求加起来达到 96GB，远远超出了物理主机实际物理内存的大小，这种情况被称为内存超售 (Memory Overcommitment)。问题是，我们能否使得这 4 台虚拟机同时运行？

我们显然无法做到将这 4 台虚拟机的所有内存数据都存储在物理内存中。在实际运行时，虽然每一台虚拟机需要的内存很大，但每一台虚拟机不会一直使用所需的最大内存数。基于此假设，可使用虚拟内存管理中的换页机制来支撑 4 台虚拟机的同时运行。具体来说，虚拟机监控器可将虚拟机中一部分不常使用的内存数据从内存中移出，保存到持久化的存储设备中，只在内存中保留每一台虚拟机正在使用或频繁使用的内存数据。这样既能在一定程度上保证虚拟机的运行性能，同时也能支撑多个虚拟机的运行。

如何在虚拟机监控器中实现换页机制？具体的步骤如下：

- 第一步：虚拟机监控器决定将某一个内存页的数据拷贝到持久性存储设备之后，首先需要将该内存页所属的虚拟机和客户物理地址信息保存起来。
- 第二步：虚拟机监控器将该页的内存数据拷贝到存储设备。

- 第三步：虚拟机监控器将此页所属虚拟机的第二阶段页表相关页表项设置为 INVALID。
- 第四步：虚拟机监控器将此页设置为“空闲”或直接分配给其他虚拟机。

如果虚拟机再次访问该页面，将触发一次第二阶段缺页异常，并唤醒虚拟机监控器。此时虚拟机监控器查询第二阶段页表，发现此客户物理地址对应的页表项为 INVALID。然后查询该页对应的换页信息，最终将数据从存储设备拷贝回内存，更新第二阶段页表并恢复虚拟机的执行。

然而，与操作系统的换页机制相比，虚拟机监控器的换页会遇到语义鸿沟（Semantic Gap）的问题。为什么会有语义鸿沟？虚拟机监控器只能“看到”寄存器或内存中的二进制存储数据，它无法将这些二进制转化成有意义的虚拟机信息。具体来说，在换页的场景下，虚拟机监控器很难知道虚拟机内的哪些页在未来一段时间内不会被使用，也很难知道哪些页在下一刻会被使用。因此，虚拟机监控器难以高效地提供透明的换页机制，从而将真正不用的内存保存到存储设备中。最极端的情况是，虚拟机监控器刚将某虚拟机的内存页换页到存储设备，不久，客户操作系统也决定将该页换页到存储设备。此时虚拟机监控器首先需要将该页的数据重新从存储设备加载回内存，并恢复第二阶段页表的映射，之后虚拟机读取该内存页数据，最后把数据保存至存储设备中。

导致这种低效率的原因是虚拟机内部缺乏和虚拟机监控器的沟通机制。一种解决方案是让客户操作系统通知虚拟机监控器哪些内存存在未来一段时间内不会被使用。具体来说，可使用半虚拟化的方式为客户操作系统提供超级调用的接口，并修改客户操作系统内核中的代码，让它不断（或在某一个特定时刻）通知虚拟机监控器哪些内存一段时间内不会被使用。通过这种方式，虚拟机监控器就可以有针对性地将这些内存页换页到存储设备中。尽管这种方式有效，但是会对操作系统内核做较大的修改，因此不是一种优雅的方法。

真正优雅的解决方案是内存气球（Memory Ballooning）。这种方案的关键思路是在客户操作系统内核插入一个伪装的驱动。该驱动不会为内核其他模块提供任何有意义的功能，它的实际目的是和虚拟机监控器互相配合，并根据虚拟机监控器的要求，不断使用客户操作系统内核的接口分配或释放内存。之后，气球驱动将分配内存的客户物理地址通知虚拟机监控器，由于这些内存页都是气球驱动独占的空闲页面，因此虚拟机监控器无须保存或恢复这些内存，可以直接将其交给其他虚拟机或其他系统服务使用。

内存气球的具体使用方式如图 14.17 所示。首先，虚拟机监控器需要给虚拟机 2 分配更多内存，因此它通知虚拟机 1 中的气球驱动，让该驱动调用客户操作系统提供的内存分配接口分配大量内存（这一步很像气球膨胀的过程）。这些内存并不会被气球驱动直接使用，在得到这些内存之后，驱动会将这些内存对应的客户物理地址发送给虚拟机监控器。此后虚拟机监控器将虚拟机 1 传递的内存映射给虚拟机 2，并通知虚拟机 2 中的气球驱动。由于此气球驱动曾经执行气球膨胀的步骤，所以它会将之前分配的内存通过接口释放给操作系统内核（这一步很像气球收缩的过程）。

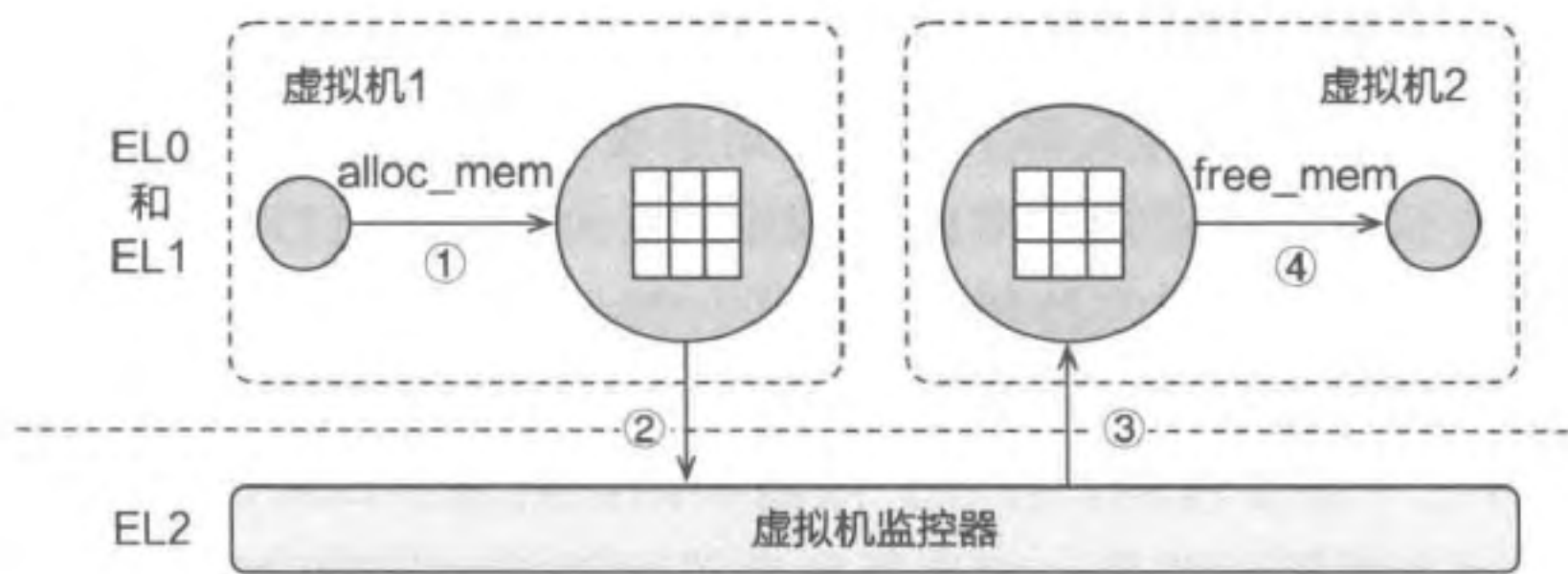


图 14.17 内存气球机制

内存气球方式之所以有效，是因为它巧妙地直接利用操作系统内核提供的内存管理接口。分配的内存都是操作系统内核维护的、未被任何模块使用的内存。这些内存存在被气球驱动得到之后，在未来一段时间内也不会被其他模块使用，因此可以安全地交给虚拟机监控器。通过这种方式，可以有效地避免之前遇到的两次换页的情况。此外，气球驱动器的写法非常简单，仅仅需要调用操作系统提供的接口，无须对操作系统内核做较大修改。因此，内存气球的方法既简单又高效。

14.4.5 小结

在本节，我们主要介绍了实现内存虚拟化的三种技术，这些技术依然可以分为两大类，如表 14.4 所示。在硬件虚拟化技术出现之前，主要通过影子页表和直接页表映射两种机制实现内存虚拟化，这两种机制在使用时也要与 14.3 节介绍的技术相配合。例如，影子页表机制需结合软件模拟的方法以便捕捉“安装页表”或“修改页表”的指令，直接页表映射机制则需与半虚拟化技术提供的超级调用接口同时使用。在硬件虚拟化技术出现之后，内存虚拟化主要依托于硬件提供的第二阶段地址翻译机制，第一阶段页表与第二阶段页表的维护被分开。

表 14.4 内存虚拟化技术总结

软件技术	硬件虚拟化技术
影子页表	第二阶段地址翻译
直接页表映射	

14.5 I/O 虚拟化

本节主要知识点

- ❑ I/O 虚拟化的功能是什么？
- ❑ 如何通过软件模拟、半虚拟化、设备直通的方式实现 I/O 虚拟化？
- ❑ 上述三种技术的优缺点分别是什么？
- ❑ SR-IOV 和 IOMMU 是如何工作的？